# Enterprise Concurrent Signal Broadcast Framework (ECSBF) Implementation

## Design Document ECSBF -Version 0.1

**Document Control:**

# Project Revision History

| Date | Version | Author | Brief Description of Changes | Approver Signature |
|---|---|---|---|---|
| 08-02-2026 | Version 0.1 | Group 4 | Initial Stage | |
| 09-02-2026 | Version 0.2 | Group 4 | Changes In Classes | |
| 09-02-2026 | Version 0.3 | Group 4 | Improved Logics | |
| 09-02-2026 | Version 0.4 | Group 4 | Improved Encryption | |
| 10-02-2026 | Version 0.5 | Group 4 | Improved Error Handling | |
| 10-02-2026 | Version 0.6 | Group 4 | Improved Node Logs | |
| 11-02-2026 | Version 0.7 | Group 4 | Improved Server Logs | |
| 11-02-2026 | Version 0.8 | Group 4 | Changes In Exception Handling | |
| 12-02-2026 | Version 0.9 | Group 4 | Changes In Server Logs | |
| 12-02-2026 | Version 1 | Group 4 | Improved Overall Logging | |

# Team Members

| | |
|---|---|
| **Souvik** | **Roy** |
| **Alisha** | **Parveen** |
| **Rittwika** | **Datta** |
| **Krithiv** | **Dharan** |

# Table of Contents

# High Level Design

## 1. Introduction

# 1.1 Purpose

The purpose of this High-Level Design (HLD) document is to provide a structured architectural overview of the Enterprise Concurrent Signal Broadcast Framework (ECSBF). This document translates the functional and non-functional requirements defined in the SRS into a clear, implementation-oriented design blueprint.

The HLD serves as a reference for developers, architects, and reviewers by describing the major system components, their responsibilities, and interactions at a high level. It also helps identify design inconsistencies, scalability concerns, and security considerations before detailed coding begins.

# 1.2 Scope

This document presents the high-level architectural design of ECSBF, focusing on how secure, concurrent, and reliable signal broadcasting is achieved across distributed nodes using a centralized Engine.

The scope of this document includes:

- High-level system architecture and component responsibilities
- Interaction between the Engine and distributed endpoint nodes
- Secure session establishment and lifecycle management
- Concurrent signal processing and broadcast mechanisms
- Observability and logging considerations

Detailed algorithmic logic, class-level implementation, and protocol-specific optimizations are addressed in the Low-Level Design (LLD) document.

# 1.3 Definitions

### 1.3.1 ECSBF (Enterprise Concurrent Signal Broadcast Framework)

A distributed communication framework designed to securely broadcast real-time signals across multiple networked nodes using a centralized concurrency Engine.

### 1.3.2 Engine

The central coordinating component of ECSBF responsible for node authentication, session management, concurrent signal handling, and unified signal broadcasting.

### 1.3.3 Node

An endpoint system or process that connects to the ECSBF Engine to send and receive broadcasted signals.

### 1.3.4 Session

A persistent logical connection established between a node and the Engine that maintains communication state and supports fault-tolerant signal exchange.

### 1.3.5 Signal

A structured data payload transmitted by a node and broadcasted by the Engine to all active nodes in the system.

## 1.4 Overview

This High-Level Design document is organized as follows:

- **Section 1 – Introduction**
  Provides the purpose, scope, key definitions, and structure of the design document.
- **Section 2 – General Description**
  Describes the overall system perspective, tools used, constraints, assumptions, and special design considerations for ECSBF.
- **Section 3 – Design Details**
  Presents the core architectural components, application architecture, data flow, interfaces, security considerations, performance expectations, and reliability aspects.

This structure ensures a logical progression from conceptual understanding to architectural clarity, enabling smooth transition into detailed design and implementation phases.

# 2. General Description

This section provides an overall description of the Enterprise Concurrent Signal Broadcast Framework (ECSBF) from a design perspective. It outlines the system's architectural viewpoint, tools and technologies, design constraints, assumptions, and special design considerations that influence the high-level structure of the framework.

## 2.1 Product Perspective

The Enterprise Concurrent Signal Broadcast Framework (ECSBF) is designed as a centralized-engine–driven distributed system. The framework consists of a Core Engine and multiple distributed endpoint nodes that communicate through secure, persistent connections.

The Engine acts as the central coordination unit and is responsible for:

- Authenticating and validating node identities
- Maintaining persistent session states
- Managing high-concurrency signal processing
- Broadcasting signals to all active nodes
- Enforcing security and access control policies

Endpoint nodes function as independent entities that can act as signal producers, consumers, or both. Each node communicates only with the Engine, ensuring controlled access, centralized observability, and simplified scalability. This architecture enables ECSBF to operate reliably in enterprise environments where real-time communication, fault tolerance, and security are critical.

## 2.2 Tools Used

The ECSBF system is designed using modern, industry-standard tools and technologies that support scalability, concurrency, and secure communication.

- **Programming Language:**
  A high-level, system-oriented programming language C++ supporting multi-threading and network programming
- **Operating System:**
  Linux-based systems (preferred), with support for Windows-based environments
- **Networking:**
  TCP/IP-based socket communication for reliable and persistent connections
- **Concurrency Model:**
  Multi-threaded processing supported by the underlying operating system
- **Security Libraries:**
  Standard cryptographic libraries for encryption and decryption of sensitive data

- **Logging and Monitoring:**
  Logging frameworks supporting multi-level diagnostic output

These tools collectively ensure that ECSBF meets enterprise-level performance, security, and maintainability requirements.

## 2.3 General Constraints

The ECSBF design is subject to the following constraints:

- All nodes must be provisioned in the Global Identity Registry prior to accessing system services
- Only authenticated and verified nodes are allowed to establish sessions
- The Engine must handle concurrent node connections without race conditions or deadlocks
- Encrypted communication must be enforced for sensitive data
- Session and identity data must be managed efficiently to avoid resource exhaustion
- Logging mechanisms must not adversely impact system performance

These constraints guide architectural decisions and ensure system robustness.

## 2.4 Assumptions

The ECSBF design is based on the following assumptions:

- The central Engine is continuously available during normal operation
- Endpoint nodes have reliable network connectivity to the Engine
- All participating systems support required cryptographic and networking libraries
- Time synchronization across nodes is reasonably accurate
- The underlying operating system supports multi-threaded execution

Any deviation from these assumptions may affect system performance or reliability.

## 2.5 Special Design Aspects

ECSBF incorporates several special design considerations to address enterprise-scale requirements:

- **High Concurrency:**
  The Engine is designed to process signals from multiple nodes simultaneously using a multi-threaded architecture.
- **Persistent Sessions:**
  Session state is preserved to support automatic recovery from transient network failures.
- **Centralized Control:**
  A single authoritative Engine simplifies identity management, monitoring, and enforcement of security policies.
- **Observability:**
  Built-in multi-level logging enables real-time monitoring, debugging, and auditing.

These design aspects ensure that ECSBF remains scalable, resilient, and secure under high operational load.

# 3. Design Details

This section presents the high-level architectural and design aspects of the Enterprise Concurrent Signal Broadcast Framework (ECSBF). It outlines the major design features, system architecture, data flow, interfaces, and quality considerations that guide the implementation of the framework.

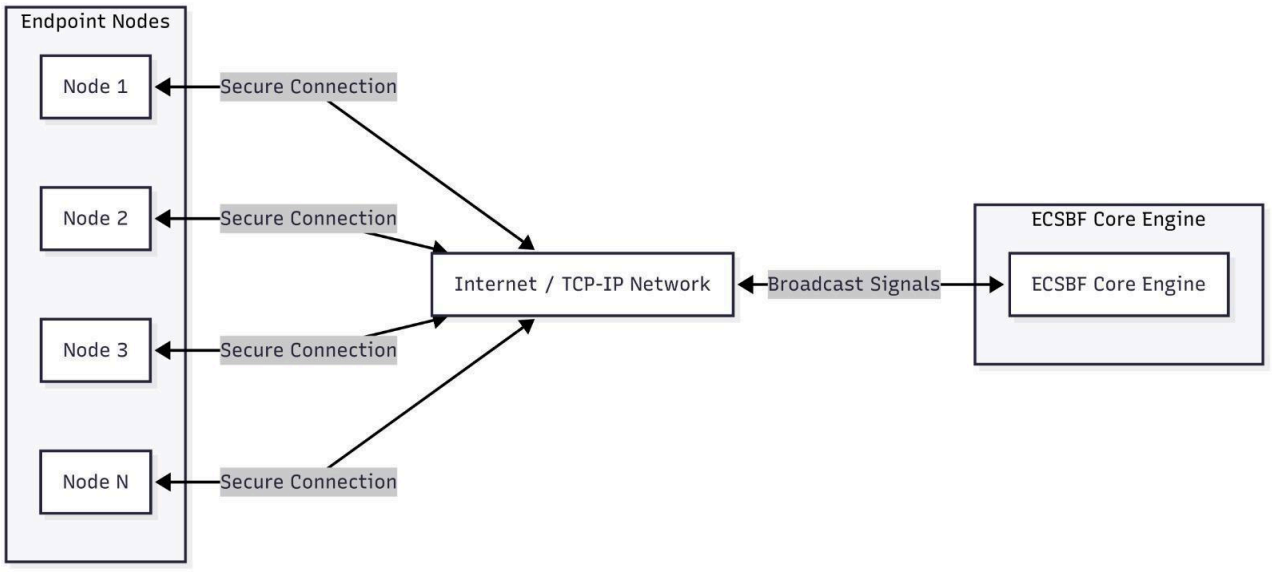
## 3.1 Main Design Features

The ECSBF design is centered around the following major features:

- Centralized Engine Architecture for controlled coordination and signal broadcasting
- Secure Node Provisioning and Identity Management
- Persistent Session Handling for fault tolerance
- High-Concurrency Processing using a multi-threaded model
- Unified Signal Broadcasting with source traceability
- Integrated Observability through multi-level diagnostic logging

These features collectively ensure scalability, security, reliability, and maintainability in enterprise environments.

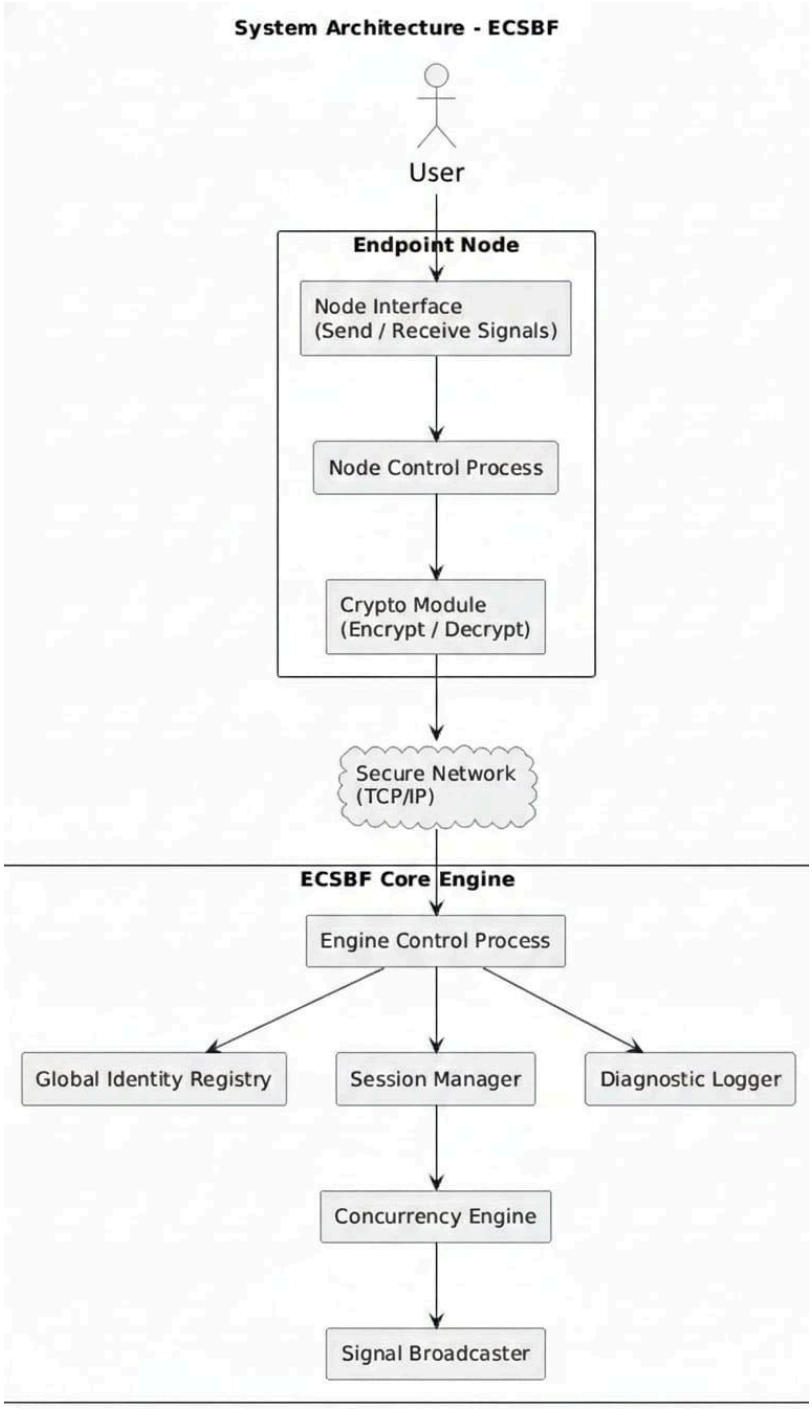
### 3.2 Application Architecture



- The application starts with a centralized core engine running on a secure engine.
- Multiple endpoint nodes connect to the core engine using TCP/IP connections.
- Each node is authenticated and a session is established before any data exchange.
- Nodes send signals to the core engine, not directly to other nodes.
- The core engine processes incoming signals concurrently.
- Processed signals are broadcasted back to all active nodes in real time.

### 3.2 .1 System Architecture

- The user starts and configures the endpoint node and ECSBF Core Engine.
- Endpoint nodes send and receive signals through a secure, encrypted TCP/IP connection.
- Each node is authenticated  using the Global Identity Registry before communication begins.
- The core engine creates and manages persistent sessions for connected nodes.
- Incoming signals are processed concurrently by the concurrency engine.
- The signal broadcaster sends validated signals to all active nodes.




### 3.2.2 System Interfaces

1.Node User Interface allows users to register, login, and send messages.

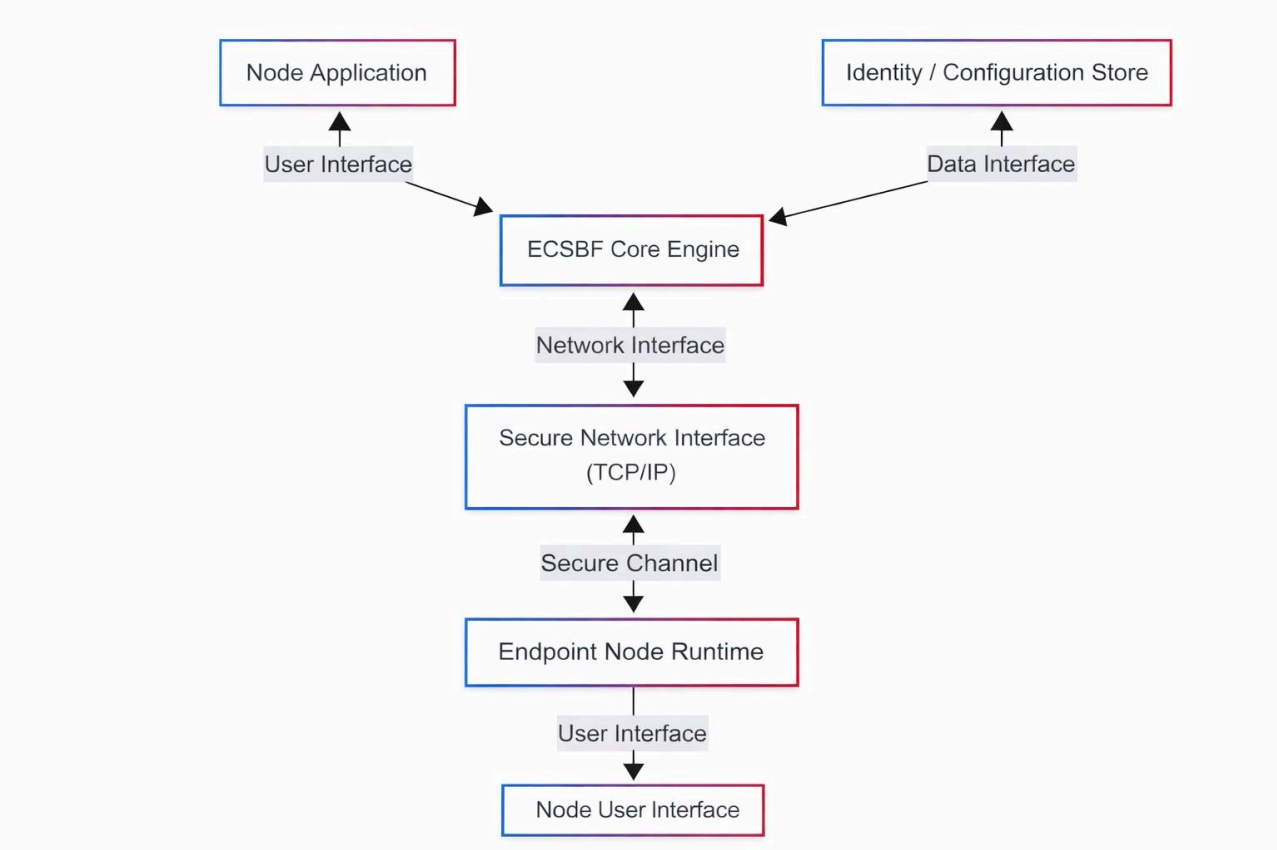
2.Endpoint Node Runtime manages socket connection and real-time communication.

3.Secure Network Interface (TCP/IP) ensures reliable client-server communication.

4.Secure Channel provides logical encryption and decryption of payloads.

5.ECSBF Core Engine handles authentication, sessions, and message broadcasting.

6.Identity Store maintains registered user credentials and node states.

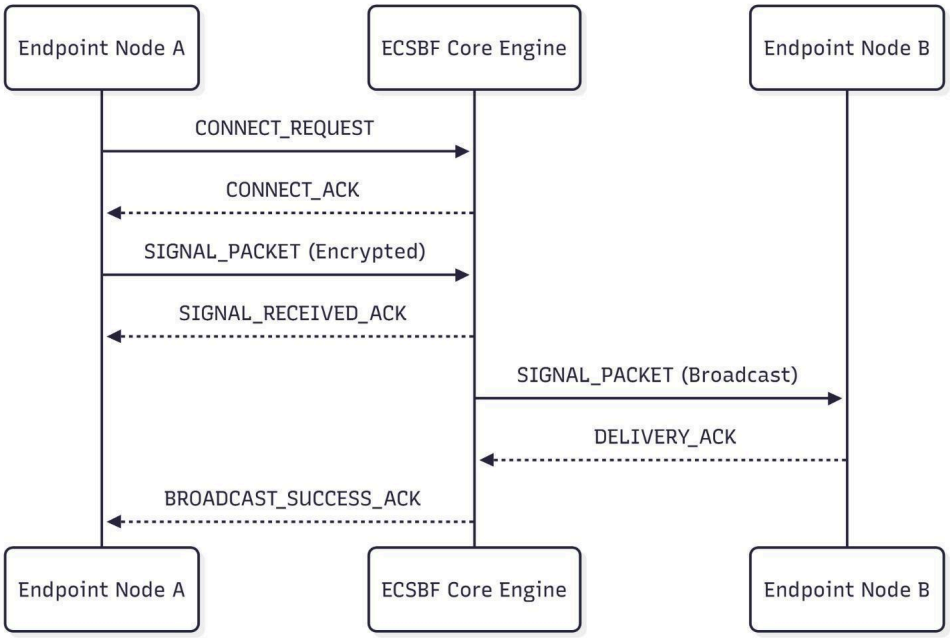



### 3.4 Data Flow Diagram

## 3.4.1 Data packet Flow Diagram

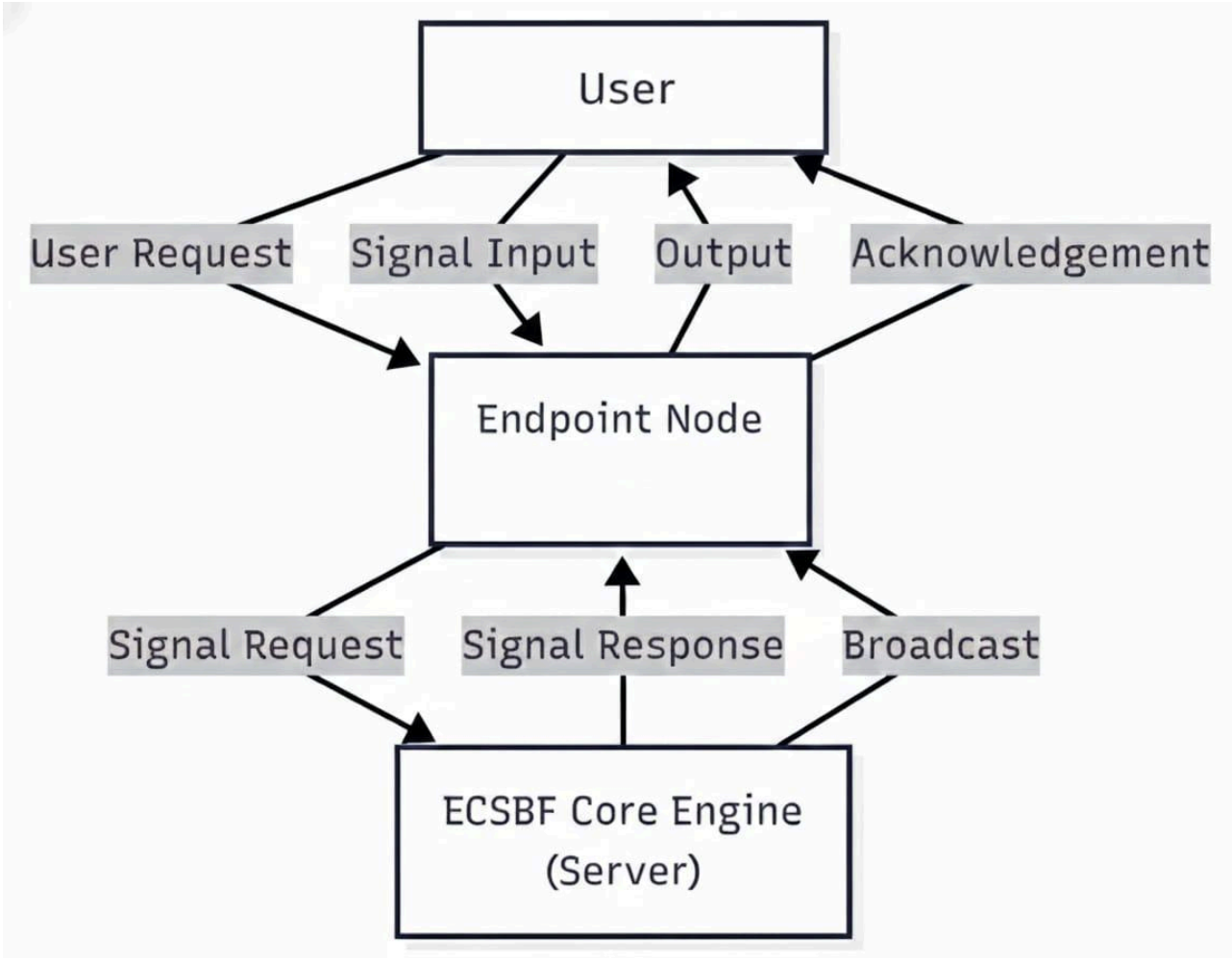
- Endpoint Node A initiates communication by sending a CONNECT_REQUEST to the ECSBF Core Engine.
- The core engine validates the request and responds with a CONNECT_ACK confirming a secure session.
- Once connected, Endpoint Node A sends an encrypted SIGNAL_PACKET to the Core Engine.
- The Core Engine acknowledges receipt using SIGNAL_RECEIVED_ACK.

- The core engine then broadcasts the signal packet to Endpoint Node B.
- Endpoint Node B receives the broadcasted signal and confirms delivery with a DELIVERY_ACK.
- After successful delivery, the Core Engine sends a BROADCAST_SUCCESS_ACK back to endpoint node A.

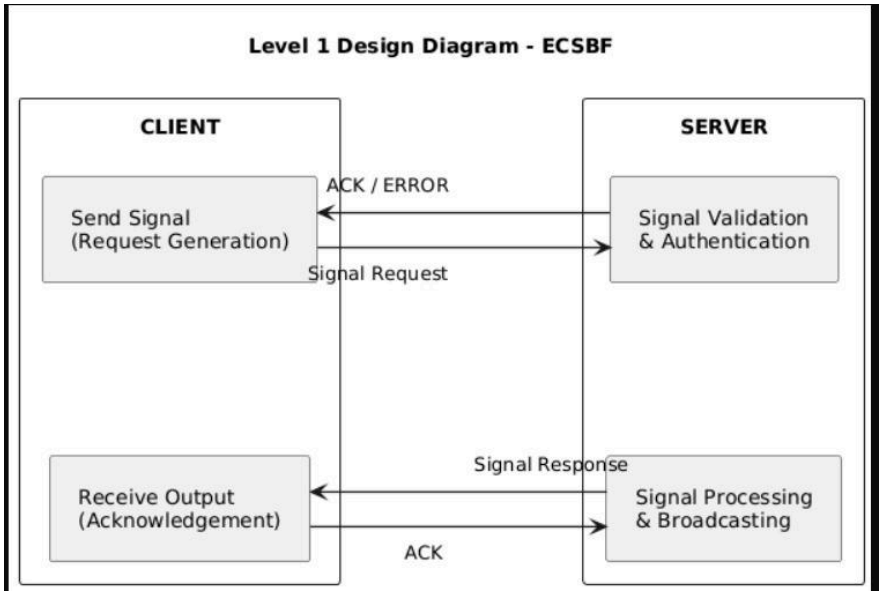



## 3.4.2 Level 0 Design Diagram

- The user provides signal input through the endpoint node interface.
- The endpoint node forwards the signal request to the ECSBF Core Engine.
- The core engine processes the signal and generates a response.
- The response or broadcast is sent back to the endpoint node.
- The endpoint node delivers output and acknowledgement to the user.

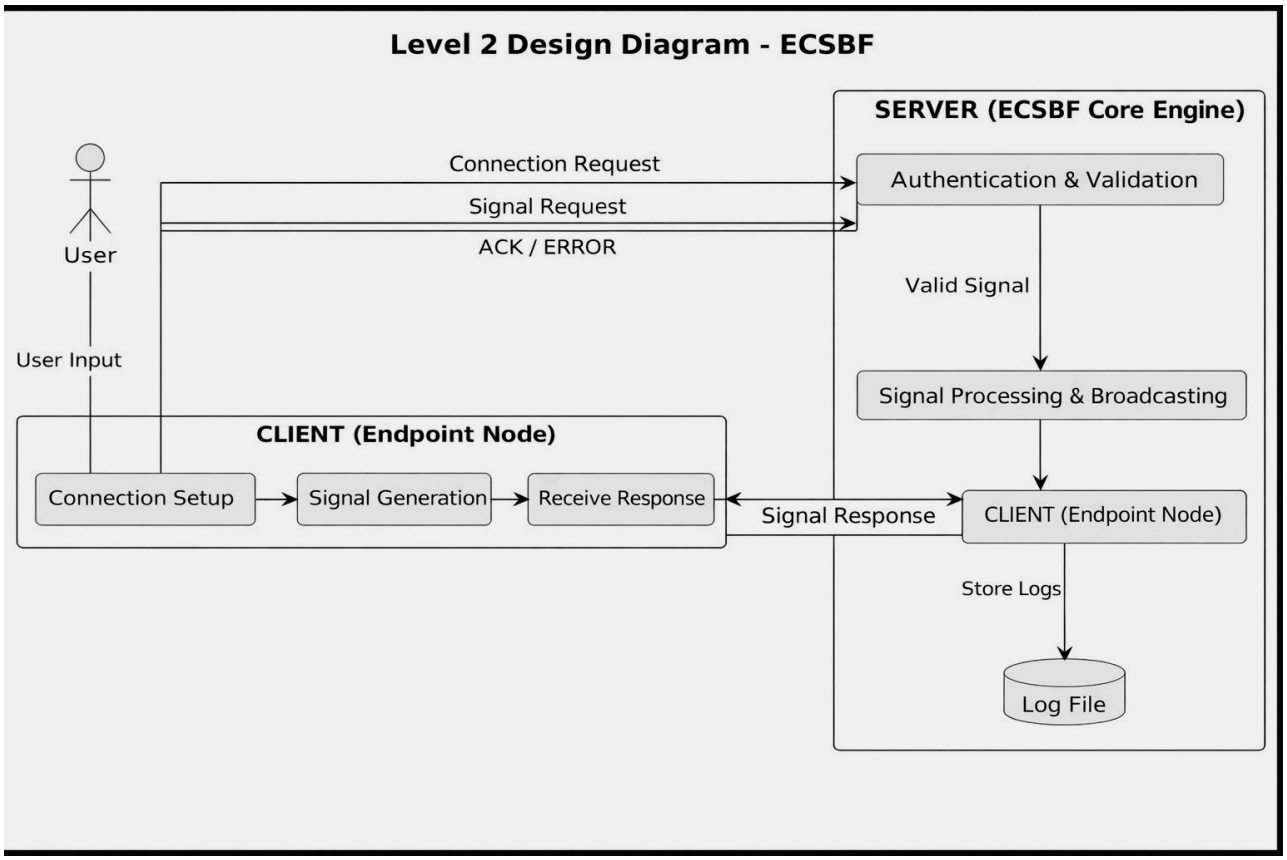



### 3.4.3 Level 1 Design Diagram



- The client generates and sends a signal request to the server.
- The server performs signal validation and authentication.
- Valid signals are processed and prepared for broadcasting.
- The server sends a signal response back to the client.
- The client receives the response and sends acknowledgment.

### 3.4.4 Level 2 Design Diagram



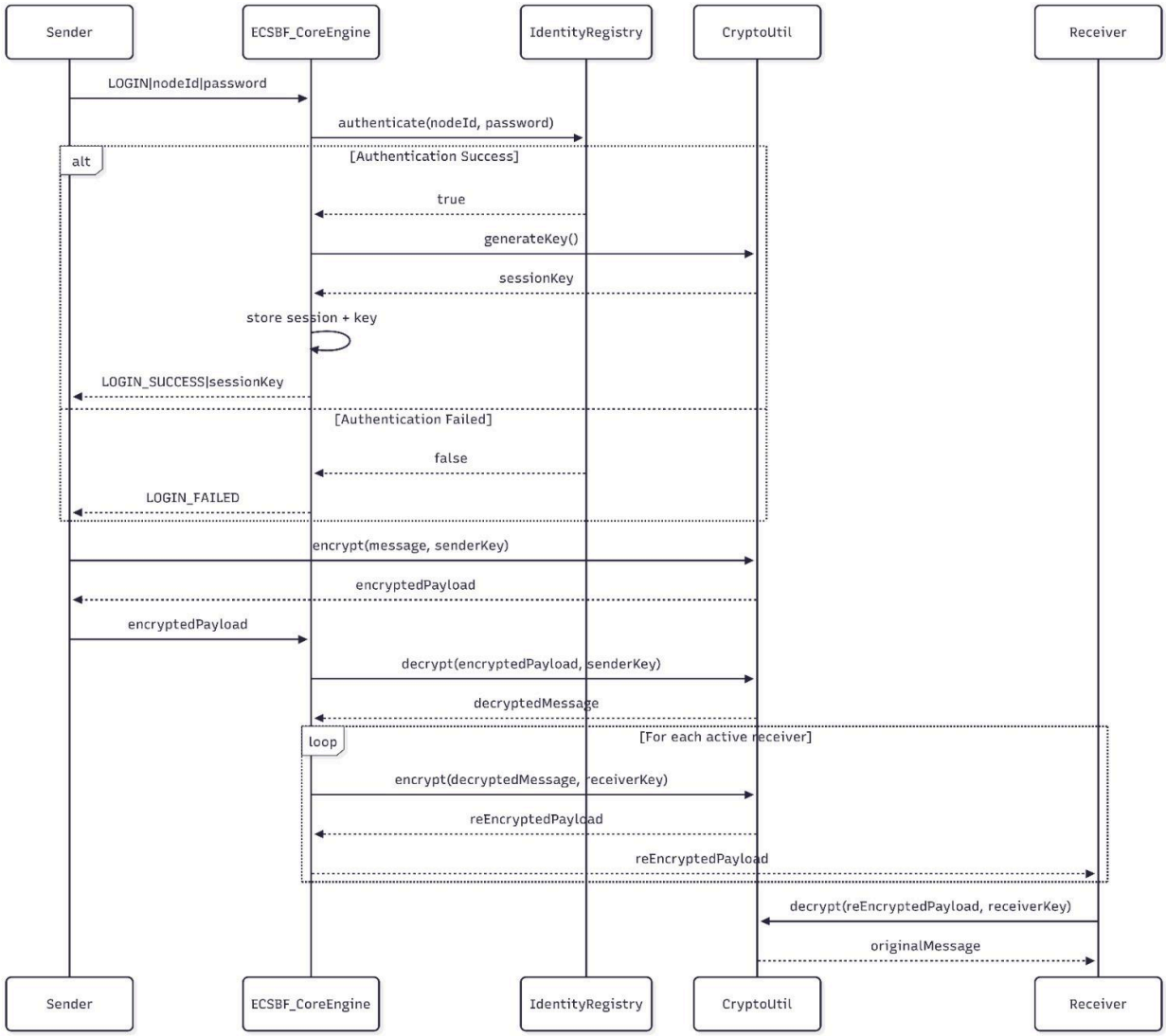

- The user initiates communication through the endpoint node.
- The client performs connection setup and signal generation.
- The ECSBF Core Engine authenticates and validates the request.
- Valid signals are processed and broadcasted to other nodes.
- Responses are returned to clients and all activities are logged.

## 3.4.5 Data Mapping Information

- The source endpoint node initializes a secure network socket and prepares session credentials.
- Credentials are encrypted and a session request is sent to the ECSBF Core Engine.
- The core engine validates the node using the Global Identity Registry.
- If authentication fails, the request is rejected and logged; otherwise a persistent session is established.
- The endpoint node encrypts the signal payload and transmits the signal packet.
- The core engine processes the signal concurrently using worker threads.
- The signal is validated, tagged for traceability and broadcasted to all active nodes.
- Destination nodes decrypt the signal, verify integrity , process the data and log errors.

## 3.4.6 Sequence Diagram

- This diagram shows how secure communication happens in the ECSBF system.
- All nodes communicate only through the Core Engine.
- A node must first login and get authenticated.
- After login, a secure session key is created.
- Messages are always sent in encrypted form.
- The Core Engine decrypts and then re-encrypts messages before sending to other nodes.
- This ensures secure, controlled, and centralized communication.




## 3.4 Files

The ECSBF framework manages a limited set of system-level files required for configuration, identity persistence, and diagnostics. No direct file transfer or file sharing occurs between nodes.

The primary files include:

- **Configuration Files:**
  Used to define engine parameters such as network ports, concurrency limits, security policies, and logging levels.

- **Global Identity Registry Data:**
  Persistent storage containing provisioned node identities and metadata.

- **Log Files:**
  System-generated logs capturing operational, diagnostic, and security-related events.

All files are centrally managed by the ECSBF Engine to ensure consistency and security.

## 3.5 User Interface

ECSBF is a backend-oriented framework and does not mandate a graphical user interface.

The supported interfaces include:

- **Command Line Interface (CLI):**
  Used by administrators to start, configure, and monitor the Engine.

- **Configuration Files:**
  Used to provision nodes and adjust runtime parameters.

- **Log Output Interface:**
  Used to observe system behavior and diagnose issues.

Any graphical dashboards or management tools are considered optional external integrations.

## 3.6 Error Handling

The ECSBF system incorporates structured error handling to ensure graceful failure and system stability.

Error conditions handled include:

- Node authentication and identity verification failures
- Session establishment and timeout errors
- Signal processing or broadcast failures
- Network or communication disruptions
- Resource exhaustion or concurrency conflicts

All errors are logged at appropriate severity levels to assist debugging and recovery.

## 3.7 Help

Help and operational guidance are provided through:

- System documentation
- Configuration and deployment guides
- Inline diagnostic logs
- Design and architecture documents

Future versions may include automated help commands or integrated documentation interfaces.

## 3.8 Performance

The ECSBF framework is designed to deliver high performance in enterprise-scale environments.

Performance considerations include:

- Low-latency signal processing and broadcasting
- Efficient handling of multiple concurrent node connections
- Minimal overhead from logging and security mechanisms
- Scalable processing capacity as node count increases

The system is optimized to support real-time communication workloads.

## 3.9 Security

Security is a foundational design aspect of ECSBF.

Key security features include:

- Mandatory node provisioning and authentication
- Secure handshake mechanisms
- Encrypted transmission of credentials and signal payloads
- Controlled session lifecycle management
- Restricted access to identity and configuration data

These measures prevent unauthorized access and protect sensitive data.

## 3.10 Reliability

ECSBF is designed to operate reliably under varying network and load conditions.

Reliability features include:

- Persistent session management
- Automatic recovery from transient network failures
- Graceful handling of node disconnections
- Isolation of failures to prevent system-wide impact

- The system ensures continuous operation whenever possible.

## 3.11 Maintainability

The ECSBF architecture emphasizes maintainability through:

- Modular component design
- Clear separation of concerns
- Well-defined interfaces between modules
- Configuration-driven behavior

These practices simplify debugging, upgrades, and long-term maintenance.

## 3.12 Portability

ECSBF is designed to be portable across multiple environments.

Portability characteristics include:

- Support for Linux and Windows platforms
- Use of standard networking and concurrency abstractions
- Minimal dependency on platform-specific features

The system can be deployed on physical machines, virtual machines, or cloud platforms.

## 3.13 Reusability

Core ECSBF components are designed for reuse in other distributed systems.

Reusable components include:

- Session management modules
- Concurrency and signal processing mechanisms
- Logging and observability infrastructure
- Identity verification workflows

This enables ECSBF concepts to be extended or integrated into other applications.

## 3.14 Application Compatibility

ECSBF is compatible with:

- Standard TCP/IP-based enterprise networks
- On-premise, cloud, and hybrid deployments
- External monitoring, logging, and orchestration tools

The framework is designed to coexist with existing enterprise infrastructure.

## 3.15 Resource Utilization

The ECSBF system is designed to use system resources efficiently.

Resource considerations include:

- Optimized CPU usage through controlled multi-threading
- Efficient memory usage for session and identity data
- Managed network bandwidth through structured signal packets
- Controlled logging to avoid unnecessary overhead

## 3.16 Major Classes

The major logical classes/components in ECSBF include:

- **Engine:**
  Central coordination and control component.
- **SessionManager:**
  Manages session creation, persistence, and termination.
- **IdentityRegistry:**
  Stores and validates node identities.
- **ConcurrencyEngine:**
  Handles parallel signal processing.
- **SignalBroadcaster:**
  Broadcasts signals to all active nodes.
- **Logger:**
  Records diagnostic and operational events.
- **NodeClient:**
  Represents an endpoint node interacting with the Engine.
- Each class has clearly defined responsibilities to ensure system clarity and robustness.

# Low Level Design

## 4.Introduction

This section introduces the Low-Level Design (LLD) for the Enterprise Concurrent Signal Broadcast Framework (ECSBF). The document provides a detailed technical view of system components, interactions, and internal workflows that are derived from the High-Level Design and Software Requirements Specification (SRS).The LLD serves as a bridge between architectural design and implementation, enabling developers to understand the internal behavior of ECSBF modules before coding begins.

### 4.1 Purpose

The purpose of this Low-Level Design document is to describe the detailed internal design of ECSBF components, focusing on module responsibilities, data flow, control logic, and interactions.

This document aims to:

- Translate high-level architectural decisions into implementable designs
- Provide clarity on internal system workflows
- Assist developers during implementation and testing
- Ensure alignment between design and ECSBF functional requirements

### 4.2 Document Conventions

The following conventions are used throughout this document:

- Headings and subheadings are formatted using bold text
- Diagrams follow standard UML notations (Sequence, Class, Use Case)
- Functional requirement references use the format ECSBF_FR_XX
- Keywords such as must, should, and may indicate requirement priority
- Technical terms are used consistently across sections
- All diagrams and descriptions adhere to standard software design documentation practices.

### 4.3 Intended Audience and Reading Suggestion

This document is intended for:

- Software developers
- System architects
- Test engineers
- Project reviewers and evaluators

Readers are expected to have a basic understanding of:

- Distributed systems
- Node–Engine architectures
- Networking fundamentals
- Concurrency and multi-threaded processing
- Secure communication principles

It is recommended to read the SRS and High-Level Design documents before proceeding with this LLD for better contextual understanding.

### 4.4 References

The following references were consulted during the design of ECSBF:

1. PlantUML Documentation – https://plantuml.com
2. OWASP Secure Design Principles – https://owasp.org

These resources support best practices in secure, scalable, and concurrent system design.

## 5. Detailed System Design

This section provides a detailed description of ECSBF's internal system design. It explains how individual modules collaborate to support secure session establishment, concurrent signal processing, and unified signal broadcasting.The detailed system design focuses on functional decomposition, module interaction, and data/control flow, forming the foundation for actual implementation.

### 5.1 Design Description

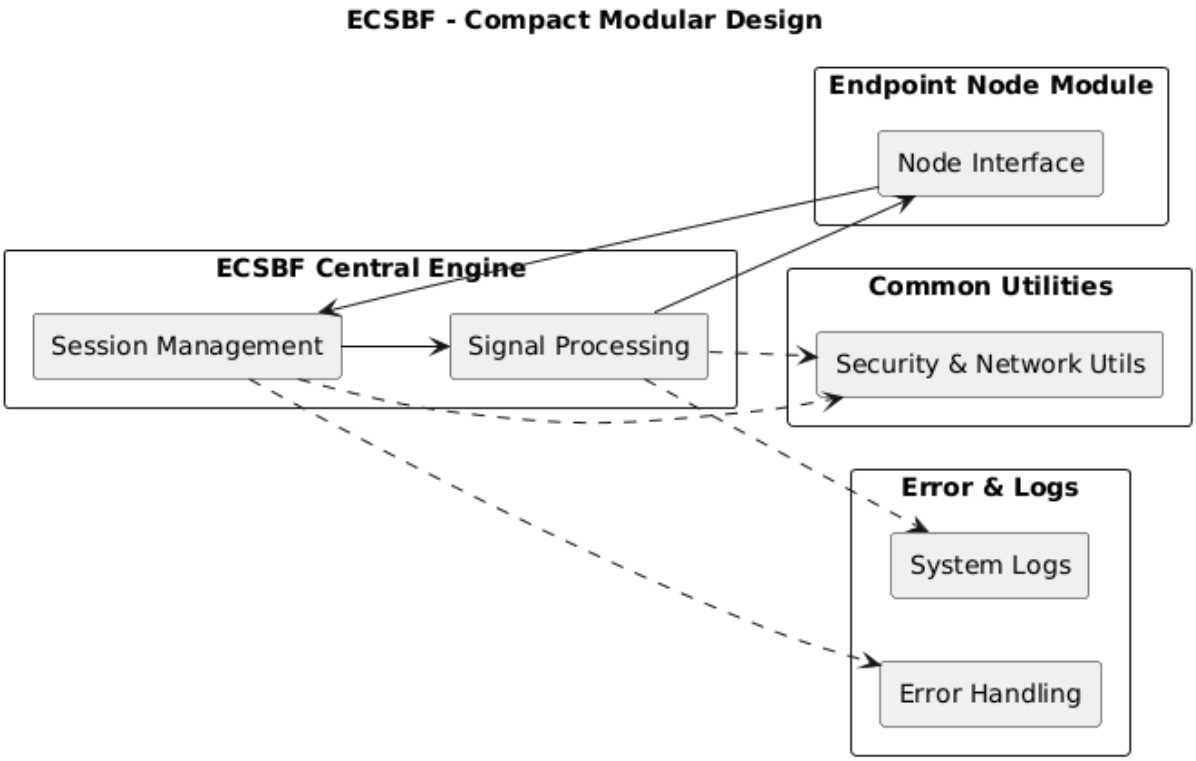
The ECSBF system follows a centralized engine-driven architecture composed of multiple interconnected modules.

At a detailed level:

- Endpoint Nodes initiate secure sessions with the ECSBF Engine
- The Engine validates node identities using the Global Identity Registry
- Persistent sessions are established and maintained
- Incoming signals are processed concurrently
- Signals are broadcasted to all active nodes with source traceability
- All system activities are logged for observability and auditing

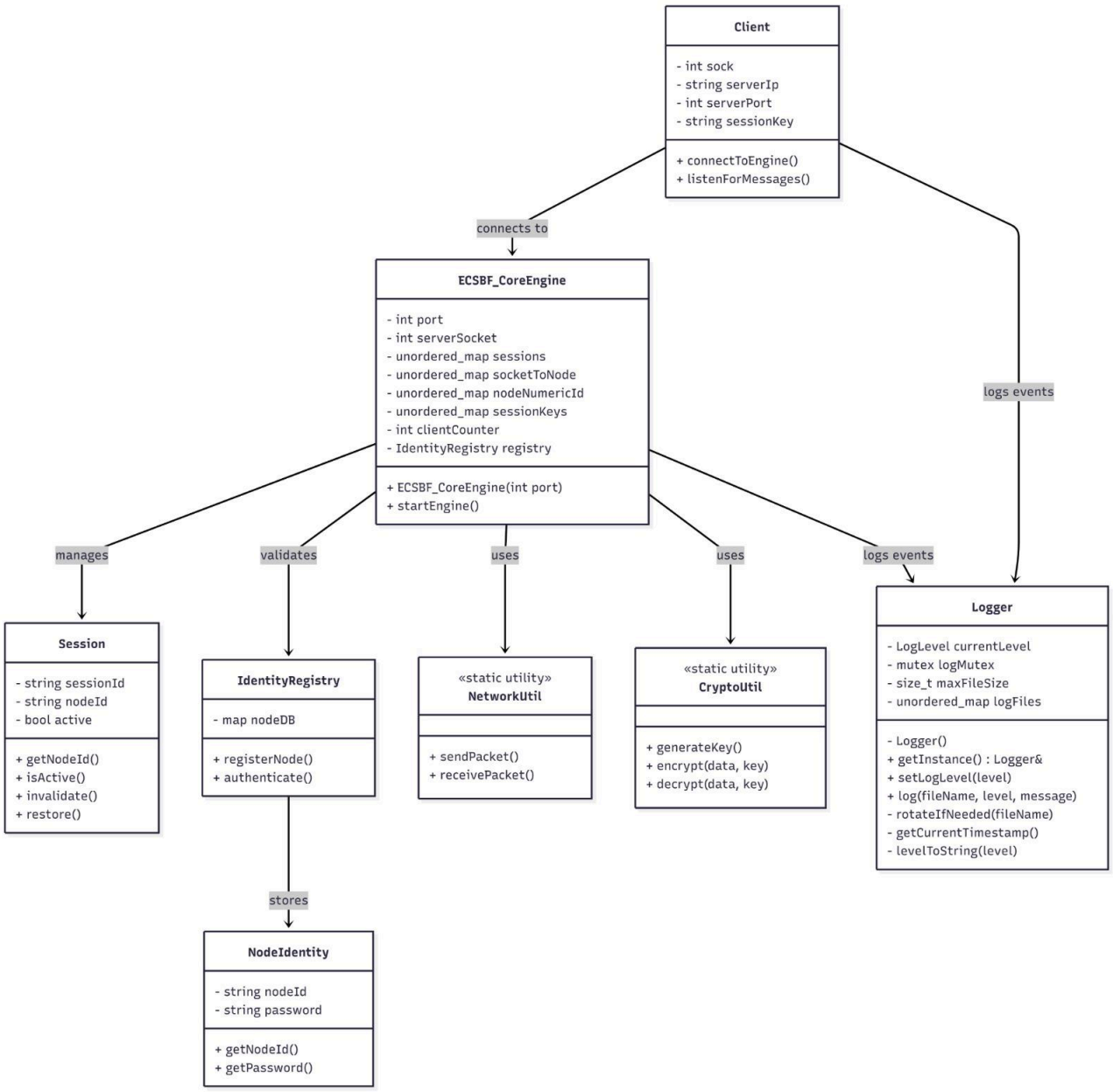
Each module is designed with a single, well-defined responsibility, ensuring scalability, maintainability, and ease of testing.

## 5.1 Modular Design



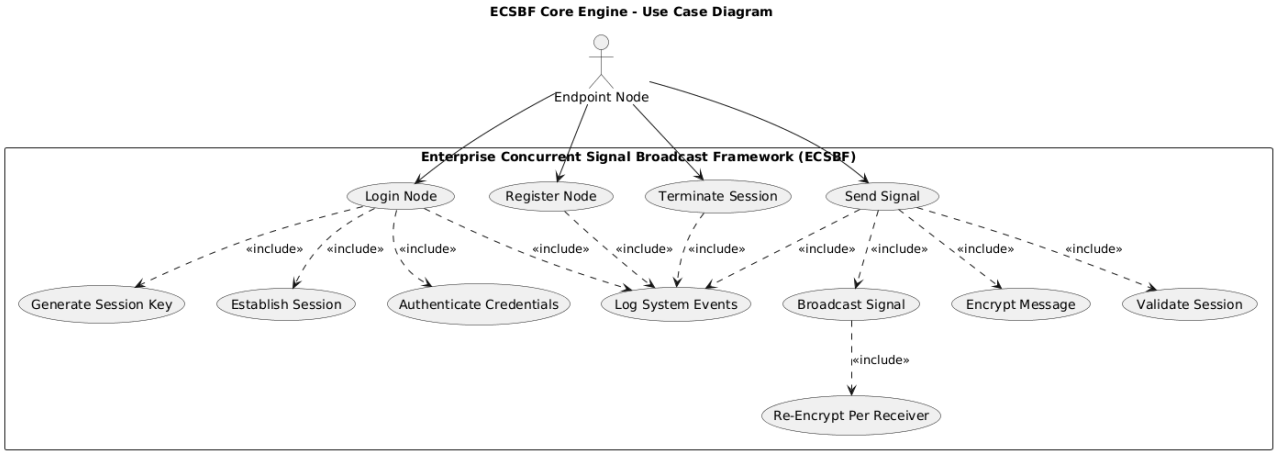

- The endpoint node module interacts with the system through the node interface.
- All node requests are routed to the ECSBF Central Engine, which acts as the core controller.
- Session Management handles node authentication, session creation and lifecycle control.
- Signal Processing manages validation, processing and preparation of signals for broadcast.
- Common utilities provide shared services like security and network operations.
- Error and logs modules record system activity and handle runtime errors.

## 5.2 Class Diagram




- This diagram shows the main classes of the ECSBF system and how they are connected.
- The Core Engine is the central class that controls the entire system.
- The Client connects to the Core Engine to send and receive signals.
- The Core Engine manages sessions for each connected node.
- It uses the Identity Registry to authenticate nodes using stored node identities.
- The Engine uses NetworkUtil to send and receive data through sockets.
- It uses CryptoUtil to encrypt and decrypt messages for secure communication.
- The Logger class records all important system events.

- **Use Case Diagram**




- ECSBF is a centralized system where multiple endpoint nodes connect to a Core Engine.
- The Core Engine controls authentication, sessions, and communication.
- A node must first register and login before using the system.
- After login, a secure session is created.
- Only active sessions can send signals.
- Messages are encrypted before being broadcast to other nodes.
- The system logs important events like login, signal sending, and session termination.
- It supports multiple nodes communicating at the same time securely.

# 5.5 Design and Implementation Constraints

The design and implementation of the Enterprise Concurrent Signal Broadcast Framework (ECSBF) are subject to the following constraints:

- **Centralized Architecture Constraint:**
  ECSBF follows a centralized engine-driven architecture. All endpoint nodes must communicate through the ECSBF Central Engine, and direct node-to-node communication is not supported.

- **Security Constraints:**

All nodes must be provisioned and authenticated before accessing system services. Encryption mechanisms must be applied to sensitive data, including credentials and signal payloads.

- **Concurrency Constraints:**
  The framework must support multiple concurrent node connections. Thread management and synchronization mechanisms are constrained by the underlying operating system and hardware resources.

- **Platform Constraints:**
  ECSBF is designed to operate on standard operating systems such as Linux and Windows. Platform-specific optimizations are avoided to ensure portability.

- **Logging and Observability Constraints:**
  Diagnostic logging must be performed without significantly degrading system performance. Logging levels must be configurable.

These constraints ensure predictable system behavior, security compliance, and maintainability.

# 5.6 User Interface

ECSBF is primarily a backend communication framework and does not require a graphical user interface for its core operation.

The supported user interaction mechanisms include:

- **Command-Line Interface (CLI):**
  Used by administrators to configure, start, and manage the ECSBF Central Engine.

- **Configuration Files:**
  Used to define system parameters such as network settings, security policies, concurrency limits, and logging levels.

- **Log-Based Interface:**
  Operational and diagnostic information is accessed through structured log outputs.

Optional graphical dashboards or monitoring tools may be integrated externally but are not part of the core ECSBF design.

# 6 Security

Security is a fundamental aspect of the ECSBF design and is enforced across all system components.

**Key security measures include:**

- **Node Provisioning and Identity Verification:**
  All endpoint nodes must be registered in the Global Identity Registry before initiating communication.

- **Secure Handshake Mechanism:**
  A secure handshake process is used to establish trusted sessions between nodes and the ECSBF Central Engine.

- **Encrypted Communication:**
  Encryption techniques are applied to credentials, session tokens, and signal payloads to prevent unauthorized access and data tampering.

- **Session Security:**
  Persistent session tokens are used to maintain secure and authenticatedcommunication.Sessions are terminated and cleaned up upon disconnection.

- **Access Control:**
  Only authenticated and authorized nodes are allowed to transmit or receive broadcast signals.

- **Security Logging and Auditing:**
  All security-relevant events are logged to support monitoring, auditing, and incident analysis.

These measures collectively protect ECSBF from unauthorized access, data breaches, and operational misuse.